NLP Lecture 2026

Traitement du Langage Naturel

Christophe Ambroise

2026-03-10

Table of contents

1 NLP Lecture 2026 **1**
1.1 Contents . . . 1
1.2 Learning objectives . . . 1

2 Words and Vectors **3**
2.1 Word-document matrix . . . 3
2.2 Word-Word matrix . . . 4
2.3 Cosine for measuring similarity . . . 4
2.4 TF-IDF: Weigthing terms in the vector . . . 5
2.5 TF-IDF: Weigthing terms in the vector . . . 5
2.6 Pointwise Mutual Information (PMI) . . . 5
2.7 Positive PMI (called PPMI) . . . 6
2.8 Applications of the tf-idf or PPMI vector models . . . 6
2.9 Latent Semantic Analysis (LSA) . . . 7
2.10 Latent Semantic Analysis . . . 7
2.11 Latent Semantic Analysis . . . 7
2.12 Latent Dirichlet Allocation (LDA) . . . 8
2.13 LDA Generative process . . . 8
2.14 Latent Dirichlet Allocation (LDA) in summary . . . 8
2.15 Latent Dirichlet Allocation (LDA) estimation . . . 9
2.16 Example of LDA in Action . . . 9
2.17 Pro and Cons . . . 10
2.18 Word2vec . . . 10
2.19 Embedding derived from classification . . . 10
2.20 From dot product to logistic regression . . . 11
2.21 Skip-gram training pairs . . . 11
2.22 Two-matrix architecture . . . 12
2.23 Final embeddings . . . 12
2.24 GloVe: from local windows to global statistics . . . 12
2.25 GloVe objective . . . 12
2.26 GloVe: meaning from co-occurrence ratios . . . 13
2.27 Word2Vec vs GloVe . . . 13

3 Exercises — LSA and Word2Vec Embeddings **15**
3.1 Links . . . 15

4 The nature of transformers **19**
4.1 The nature of transformers . . . 21

4.2 Optimized criteria . . . 21
4.3 Masked Language Modeling (MLM) . . . 22
4.4 Conditional Language Modeling / Translation Loss . . . 22
4.5 Initial embedding . . . 22
4.6 A minimal self-attention architecture for decoder . . . 23
4.7 Attention and matrix notation . . . 23
4.8 Ilustration of self-Attention . . . 24
4.9 Interpretation of Q, K, V . . . 24
4.10 Important details . . . 24
4.11 Positional encoding . . . 25
4.12 Feed Forward Neural Network . . . 25
4.13 Skip connection (original paper "Deep Residual Learning for Image Recognition" 2015 by He et al.) . . . 26
4.14 Multi-head attention . . . 26
4.15 Fine tuning . . . 27

5 Exercises — GPT from scratch 29
5.1 Links . . . 29
5.2 Supplementary links . . . 29

List of Figures

4.1 History of AI leading to chatGPT 19
4.2 The nature of transformer 21
4.3 Initial embedding 23
4.4 Self-attention block in auto-encoder 24
4.5 Skip connection 26
4.6 ChatGPT fine tuning (from A. Karpathy lecture) 27

List of Tables

2.1 Term-document matrix — S1: *"word embeddings represent meaning"* · S2: *"deep neural networks learn representations"* · S3: *"language models process text"* · S4: *"word vectors capture context"* · S5: *"neural language models predict"* 3
2.2 Word-word co-occurrence matrix (window $L = 1$) 4
2.3 TF-IDF weighted term-document matrix — $\mathrm{tf} = \log_{10}(1+1) = 0.30$ for all present terms (each appears once per sentence) 5
2.4 PPMI matrix — all other co-occurrences have PMI ≤ 0, set to 0 by PPMI 6

Chapter 1

NLP Lecture 2026

Natural Language Processing

This document covers the fundamentals of NLP, from word embeddings to Transformer architectures, with practical exercises based on nanoGPT.

1.1 Contents

1. **Word Embeddings** - Word vectors, co-occurrence matrices, TF-IDF, PMI, LSA, LDA, and Word2Vec
2. **Transformers** - Attention architecture, contextual embeddings, and language models
3. **Exercises** - Practical implementation with nanoGPT and key concepts

1.2 Learning objectives

- Understand attention mechanisms and Transformers
- Master different approaches to word embeddings
- Implement the concepts with practical code

Course 2026

Chapter 2

Words and Vectors

Vector or distributional models of meaning are generally based on a co-occurrence matrix

2.0.1 Two common co-occurrence matrix

- the word-document matrix
- the word-word matrix.

2.1 Word-document matrix

In a Word-document matrix, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

Corpus of 5 short sentences about NLP (used as a running example throughout this chapter):

Table 2.1: Term-document matrix — S1: *"word embeddings represent meaning"* · S2: *"deep neural networks learn representations"* · S3: *"language models process text"* · S4: *"word vectors capture context"* · S5: *"neural language models predict"*

	S1	S2	S3	S4	S5
word	1	0	0	1	0
embeddings	1	0	0	0	0
vectors	0	0	0	1	0
neural	0	1	0	0	1
language	0	0	1	0	1
models	0	0	1	0	1
context	0	0	0	1	0
meaning	1	0	0	0	0

> **Note**
>
> "embeddings" (S1) and "vectors" (S4) never appear in the same document, yet both co-occur with "word". LSA will detect this indirect synonymy.

2.2 Word-Word matrix

In the Word-context matrix the columns are labeled by words rather than documents.

- each cell records the number of times the row (target) word and the column (context) word co-occur in some context in some training corpus.

2.2.1 Context

- a context could be the document, in which case the cell represents the number of times the two words appear in the same document.
- smaller contexts are common: generally a window around the word

Word-word matrix for the same corpus (window $L = 1$, symmetric counts):

Table 2.2: Word-word co-occurrence matrix (window $L = 1$)

	word	embed-dings	vectors	represent	meaning	capture	context
word	—	1	1	0	0	0	0
em-bed-dings	1	—	0	1	0	0	0
vec-tors	1	0	—	0	0	1	0
rep-re-sent	0	1	0	—	1	0	0
mean-ing	0	0	0	1	—	0	0
cap-ture	0	0	1	0	0	—	1
con-text	0	0	0	0	0	1	—

> **Note**
>
> "embeddings" and "vectors" share the same context word ("word") → identical column profiles → cosine similarity = 1, even though they never appear in the same sentence.

2.3 Cosine for measuring similarity

The cosine similarity metric (or empirical correlation) between two word vectors v and w (lines of a word-document or word-word matrix)

$$\cos(v, w) = \frac{v^T w}{\|v\|\|w\|}$$

2.4 TF-IDF: Weigthing terms in the vector

2.4.1 Term frequency (tf)

- Raw frequency $tf_{t,d} = count(t, d)$ is very skewed and not very discriminative.
- Usually squashed by using the $log10$: a word appearing 100 times in a document doesn't make that word 100 times more relevant

$$\mathrm{tf}_{t,d} = \log_{10}(count(t, d) + 1)$$

2.4.2 Inverse document frequency

The second factor in tf-idf is used to give a higher weight to words that occur only in a few documents.

$$idf_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

where N is the total number of documents, and df_t is the number of documents in which term t occurs.

2.5 TF-IDF: Weigthing terms in the vector

The tf-idf weighted value $w_{t,d}$ for word t in document d thus combines term frequency $tf_{t,d}$ with idf:

$$w_{t,d} = tf_{t,d} \times idf_t$$

Applied to our running corpus ($N = 5$ documents):

Table 2.3: TF-IDF weighted term-document matrix — $\mathrm{tf} = \log_{10}(1 + 1) = 0.30$ for all present terms (each appears once per sentence)

	S1	S2	S3	S4	S5	idf
word	0.12	0	0	0.12	0	0.40
embeddings	**0.21**	0	0	0	0	0.70
vectors	0	0	0	**0.21**	0	0.70
neural	0	0.12	0	0	0.12	0.40
language	0	0	0.12	0	0.12	0.40
models	0	0	0.12	0	0.12	0.40
context	0	0	0	**0.21**	0	0.70
meaning	**0.21**	0	0	0	0	0.70

Words appearing in only one document ("embeddings", "vectors", "context", "meaning") receive the highest weight 0.21; words shared across two documents ("word", "neural", "language", "models") receive 0.12.

2.6 Pointwise Mutual Information (PMI)

An alternative weighting function to tf-idf, PPMI (positive pointwise mutual information), is used for **term(target)-term(context)-matrices**.

- Target word, word we focus on.

- Context words surrounding the target word

The pointwise mutual information between a target word w and a context word c (Church and Hanks 1989, Church and Hanks 1990) is defined as:

$$PMI(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

2.7 Positive PMI (called PPMI)

Negative PMI values (which imply things are co-occurring less often than we would expect by chance) tend to be unreliable unless our corpora are enormous.

2.7.1 Positive PMI

replaces all negative PMI values with zero (Church and Hanks 1989, Dagan et al. 1993, Niwa and Nitta 1994)

$$PPMI(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0)$$

Applied to our word-word matrix (16 context pairs total, window $L = 1$):

$$\text{PMI}(\text{embeddings}, \text{word}) = \log_2 \frac{P(\text{emb}, \text{word})}{P(\text{emb}) \cdot P(\text{word})} = \log_2 \frac{1/16}{(2/16)(2/16)} = \log_2 4 = 2.0$$

Table 2.4: PPMI matrix — all other co-occurrences have PMI ≤ 0, set to 0 by PPMI

	word	embeddings	vectors	represent	meaning
word	—	**2.0**	**2.0**	0	0
embeddings	**2.0**	—	0	**2.0**	0
vectors	**2.0**	0	—	0	0
represent	0	**2.0**	0	—	**2.0**
meaning	0	0	0	**2.0**	—

> **Note**
>
> "embeddings" and "vectors" have **identical PPMI profiles**: both associate with "word" only (PMI = 2.0) and with nothing else. PPMI already encodes their synonymy before any SVD.

2.8 Applications of the tf-idf or PPMI vector models

2.8.1 Principle

- Computing two documents (or word) similarity after transformation.
- Given two documents d_1 and d_2, the similarity is $\cos(d_1, d_2)$.

2.8.2 Applications

- **Documents** : information retrieval, plagiarism detection, news recommender systems, and even for digital humanities tasks like comparing different versions of a text to see which are similar to each other.
- **Words** : finding word paraphrases, tracking changes in word meaning, or automatically discovering meanings of words in different corpora.

2.9 Latent Semantic Analysis (LSA)

2.9.1 Goal and assumptions

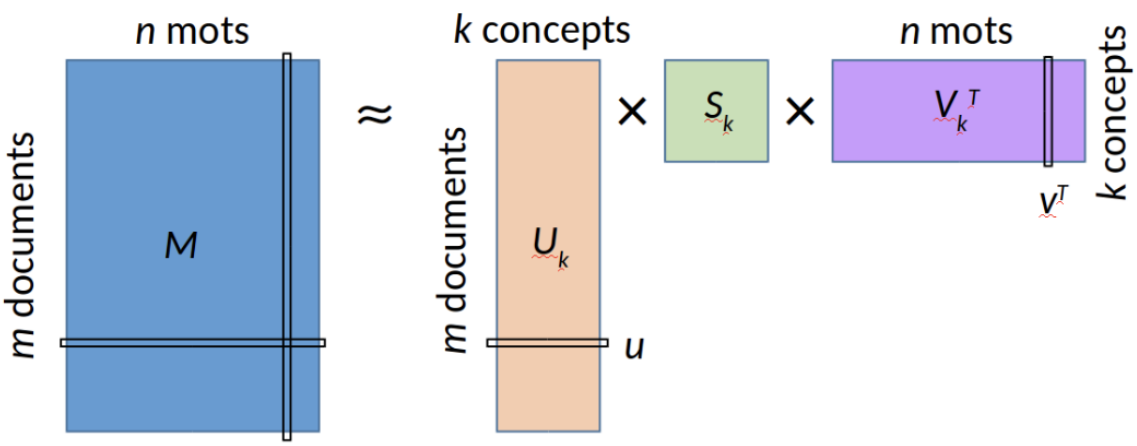
- extracting and representing the underlying meaning of words in a corpus of texts.
- words that occur in similar contexts have similar meanings.

2.9.2 Principle

LSA uses singular value decomposition (SVD) to identify latent, or hidden, patterns in word co-occurrence data.

2.10 Latent Semantic Analysis

LSA uses SVD on the matrix of word-document matrix to identify the latent concepts (contexts, topics, ...) that underlie the relationships between words in the corpus: $W = USV^T$ with classical low rank approximation $U_k S_k V_k^T$



2.11 Latent Semantic Analysis

- The values on the main diagonal of S_k indicate the 'importance' of each of the k main 'latent concepts' (or factors).
- For each of the document, the corresponding row of U_k allows us to see which concepts are present and with what weights.
- For each of the concept, the associated column of V_k indicates which terms form the concept (and with what weights).

Calculating the similarity between a term and a document involves choosing:

- the row $\mathbf{u}$ corresponding to the document in the matrix $\mathbf{U}_k$,
- the row $\mathbf{v}$ corresponding to the term in the matrix $\mathbf{V}_k$,
- and then computing the product $\mathbf{u}\mathbf{S}_k\mathbf{v}^T$.

To determine the similarity of a term to each of the documents

$$\mathbf{U}_k\mathbf{S}_k\mathbf{v}^T$$

Conversely, we can determine the similarities between a document $\mathbf{u}$ and all the terms through

$$\mathbf{u}\mathbf{S}_k\mathbf{V}_k^T$$

2.12 Latent Dirichlet Allocation (LDA)

2.12.1 History

In the context of population genetics, LDA was proposed by J. K. Pritchard, M. Stephens and P. Donnelly in 2000.

LDA was applied in machine learning by David Blei, Andrew Ng and Michael I. Jordan in 2003.

2.12.2 Principle

Latent Dirichlet Allocation (LDA) is a probabilistic generative model that assumes that

- each document is generated by a mixture of latent topics
- each topic is generated by a mixture of words from the vocabulary.

Documents are represented as random mixtures over latent topics, where each topic is characterized by a distribution over all the words.

2.13 LDA Generative process

For a corpus D consisting of M documents

1. Choose topic parameter vector for document i: $\theta_i \sim \mathrm{Dir}(\alpha)$, where $i \in \{1, \dots, M\}$ and $\mathrm{Dir}(\alpha)$ is a Dirichlet distribution with a symmetric parameter α which typically is sparse (α).
2. Choose the word parameters vector of topic k: $\varphi_k \sim \mathrm{Dir}(\beta)$, where $k \in \{1, \dots, K\}$ and β typically is sparse
3. For each of the word positions i, j, where $i \in \{1, \dots, M\}$, and $j \in \{1, \dots, N_i\}$

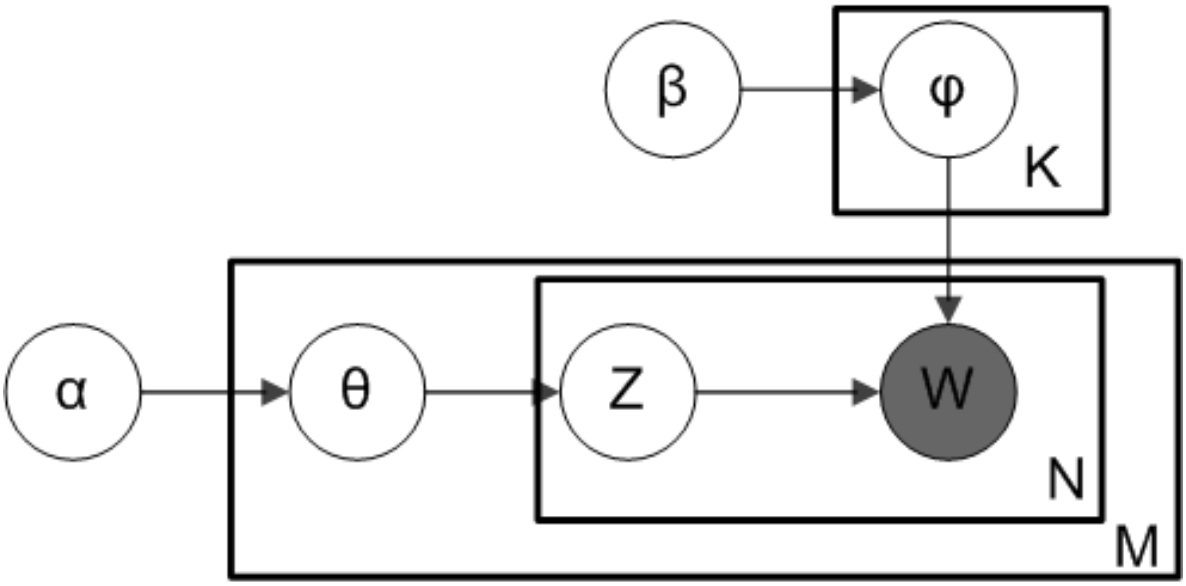
a. Choose a topic $z_{i,j} \sim \mathrm{Cat}(\theta_i)$.
b. Choose a word $w_{i,j} \sim \mathrm{Cat}(\varphi_{z_{i,j}})$.

2.14 Latent Dirichlet Allocation (LDA) in summary

$$\varphi_{k=1\dots K} \sim \mathrm{Dirichlet}_V(\beta) \tag{2.1}$$
$$\theta_{d=1\dots M} \sim \mathrm{Dirichlet}_K(\alpha) \tag{2.2}$$
$$z_{d=1\dots M, w=1\dots N_d} \sim \mathrm{Categorical}_K(\theta_d) \tag{2.3}$$
$$w_{d=1\dots M, w=1\dots N_d} \sim \mathrm{Categorical}_V(\varphi_{z_{dw}}) \tag{2.4}$$

$$P(W, Z, \theta, \varphi; \alpha, \beta) = \prod_{i=1}^{K} P(\varphi_i; \beta) \prod_{j=1}^{M} P(\theta_j; \alpha) \prod_{t=1}^{N} P(Z_{j,t} \mid \theta_j) P(W_{j,t} \mid \varphi_{Z_{j,t}}),$$

2.15 Latent Dirichlet Allocation (LDA) estimation



Estimating the parameters could be achieved through Variationnal Bayes EM algorithm.

2.16 Example of LDA in Action

Corpus of Three Documents

1. "cats dogs pets love"
2. "dogs bark loud outside"
3. "politics government election law"

Step 1: Identify Topics

After running LDA, it may discover two topics (dist. over words):

- Topic 1 (Pets): {cats, dogs, pets, bark, love}
- Topic 2 (Politics): {politics, government, election, law}

Step 2: Assign Topics to Documents

- Document 1 -> 90% Topic 1, 10% Topic 2
- Document 2 -> 80% Topic 1, 20% Topic 2
- Document 3 -> 95% Topic 2, 5% Topic 1

2.17 Pro and Cons

2.17.1 Advantages of LDA

- Unsupervised Learning: No labeled data needed.
- Interpretable Topics: Extracts meaningful topics from text.

2.17.2 Limitations of LDA

- Fixed Number of Topics: Must specify the number of topics beforehand.
- Bag-of-Words Assumption: Ignores word order and context.
- Computationally Expensive: Requires approximation methods for inference.

2.17.3 Applications of LDA

- Topic Modeling: Organizing large text corpora (news articles, research papers).
- Recommendation Systems: Suggesting articles based on topic similarity.
- Text Classification: Assigning labels based on topic distributions.
- Social Media Analysis: Identifying trending topics in posts.

2.18 Word2vec

2.18.1 Embeddings

- more powerful word representation
- short dense vectors: d ranging from 50-1000, rather than the much larger vocabulary size $|V|$ or number of documents D
- dense vectors work better in every NLP task than sparse vectors

2.18.2 Skip-gram with negative sampling

- The skip-gram algorithm is one of two algorithms in a software package called word2vec
- The other algorithm is CBOW (continuous bag of words)

2.19 Embedding derived from classification

2.19.1 Data : words and their context (neighboring words ($\pm L$)

… lemon, a [tablespoon of apricot jam, a] pinch … c1 c2 w c3 c4

2.19.2 Classification task

Given a tuple $(w, c) \in \mathbb{R}^d \times R^d$ of a target word w paired with a candidate context word c (for example (apricot, jam), or perhaps (apricot, aardvark)) return the probability that c is a real context word (true for jam, false for aardvark):

$$P(y_{wc} = 1|w, c) = 1 - P(y_{wc} = 0|w, c)$$

2.19.3 Similarity

we rely on the intuition that two vectors are similar if they have a high dot product (after all, cosine is just a normalized dot product). In other words:

$$Similarity(w, c) = c^T w$$

2.20 From dot product to logistic regression

2.20.1 Logit

$$P(y_{wc} = 1|w, c) = \frac{1}{1 + \exp(-c^T w)}$$

2.20.2 Criterion

The criterion is a kind of likelihood including positive examples (observed association) and negative examples (non observed associations). In fact skip-gram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter k).

$$L = \sum_{w,c:y_{wc}=1} \log P(y_{wc} = 1|w, c) + \sum_{w',c':y_{w'c'}=0} \log P(y_{w'c'} = 0|w', c')$$

2.21 Skip-gram training pairs

Sentence: *"word **embeddings** represent meaning"* — target = **embeddings**, window $L = 2$

Positive pairs (real context, $y = 1$):

target	context	label
embeddings	word	1
embeddings	represent	1
embeddings	meaning	1

Negative samples ($k = 2$ per positive, sampled $\propto P(w)^{3/4}$):

target	noise word	label
embeddings	neural	0
embeddings	deep	0
embeddings	process	0

The model is trained to push $c^T_{\text{word}} w_{\text{emb}} \to +\infty$ and $c^T_{\text{neural}} w_{\text{emb}} \to -\infty$.

After training on all sentences: $w_{\text{embeddings}} \approx w_{\text{vectors}}$ because both always appear next to "word".

2.22 Two-matrix architecture

Skip-gram maintains two $|V| \times d$ embedding matrices:

$$W = \begin{bmatrix} \leftarrow w_{\text{word}} \rightarrow \\ \leftarrow w_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix} \qquad C = \begin{bmatrix} \leftarrow c_{\text{word}} \rightarrow \\ \leftarrow c_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix}$$

- **Target matrix** W: used when the word is the focus of a training pair.
- **Context matrix** C: used when the word appears as context.

The logit for pair (w, c) is a single dot product: $c^T w$.

> **Note**
>
> The window size L controls the nature of the learned similarity: $L \leq 2$ captures syntactic relations · $L \geq 5$ captures topical/semantic relations.

2.23 Final embeddings

- skip-gram outputs the target matrix W and the context matrix C.
- It's common to just add them together, representing word i with the vector $w_i + c_i$.
- Alternatively we can throw away the C matrix and just represent each word i by the vector w_i.
- As with the simple count-based methods like tf-idf, the context window size L affects the performance of skip-gram embeddings, and experiments often tune the parameter L on a devset.

2.24 GloVe: from local windows to global statistics

Word2Vec learns from local context windows — each gradient step sees one (target, context) pair. The global co-occurrence structure of the corpus emerges only implicitly after many steps.

GloVe (Pennington et al., 2014) makes global statistics *explicit*:

1. Build the full co-occurrence matrix $\mathbf{X}$ where X_{ij} = number of times word j appears in the context of word i across the entire corpus.
2. Directly fit word vectors to this matrix.

> **Note**
>
> On our running corpus, $X_{\text{embeddings,word}} = 1$ and $X_{\text{vectors,word}} = 1$ — GloVe sees this symmetry globally and will assign similar vectors to both words.

2.25 GloVe objective

Learn $w_i, \tilde{w}_j \in \mathbb{R}^d$ and scalar biases $b_i, \tilde{b}_j$ to minimise:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Weighting function — downweights rare pairs, caps frequent ones:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & x \geq x_{\max} \end{cases} \qquad x_{\max} = 100,\ \alpha = 0.75$$

Intuition: the dot product of two word vectors should predict their log co-occurrence count: $w_i^T \tilde{w}_j \approx \log X_{ij}$.

2.26 GloVe: meaning from co-occurrence ratios

The key insight is that **ratios** of co-occurrence probabilities encode semantic relations:

	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	ratio
$k = \text{solid}$	large	small	$\gg 1$
$k = \text{gas}$	small	large	$\ll 1$
$k = \text{water}$	large	large	≈ 1
$k = \text{fashion}$	small	small	≈ 1

GloVe vectors encode these ratios directly:

$$w_i^T \tilde{w}_k - w_j^T \tilde{w}_k \approx \log \frac{P(k \mid i)}{P(k \mid j)}$$

This explains why linear analogy arithmetic ($w_{\text{king}} - w_{\text{man}} + w_{\text{woman}} \approx w_{\text{queen}}$) works: the vector difference encodes a log probability ratio.

2.27 Word2Vec vs GloVe

	Word2Vec (SGNS)	GloVe
Training signal	Local windows, one pair at a time	Global co-occurrence matrix $\mathbf{X}$
Objective	Logistic classification	Weighted least squares
Scalability	Streams corpus (SGD)	Requires building $\mathbf{X}$ first
Small corpora	Better (no need to store $\mathbf{X}$)	Worse
Interpretability	Implicit	Explicit ($\log X_{ij}$ target)
Analogy accuracy	High	High (comparable)

Pre-trained vectors are available for both: `word2vec-google-news-300` (100B tokens), `glove-wiki-gigaword-{50,100,200,300}d` (6B tokens).

> **!** Important
>
> **Shared limitation**: both methods produce **one static vector per word** regardless of context. The word *bank* gets the same representation whether it means a riverbank or a financial institution. Contextualised embeddings — ELMo, BERT, GPT — overcome this by making representations

depend on the full sentence context.

Chapter 3

Exercises — LSA and Word2Vec Embeddings

These exercises explore two complementary approaches to building word representations:

- **Latent Semantic Analysis (LSA)** — a classical count-based method using SVD on a term-document matrix
- **Word2Vec** — a neural prediction-based method using skip-gram with negative sampling

Both methods were introduced in the lecture (`word-embedding.qmd`). The practical work is in `TP/lsa_word2vec.ipynb`.

3.1 Links

- Original LSA paper — Deerwester et al. (1990)
- Word2Vec paper — Mikolov et al. (2013)
- GloVe paper — Pennington et al. (2014)
- Gensim documentation — Word2Vec and pre-trained vectors
- Jurafsky & Martin, Chapter 6 — Vector semantics and embeddings

---

3.1.1 1) Term-document matrix and cosine similarity

Goal: Build a count matrix from a toy corpus and measure word similarity by cosine distance.

Key ideas

- A corpus of D documents with a vocabulary of $|V|$ words defines a $|V| \times D$ term-document matrix.
- Raw counts can be weighted (TF-IDF) to reduce the influence of frequent uninformative words.
- Cosine similarity $\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v}^T\mathbf{w}}{\|\mathbf{v}\|\|\mathbf{w}\|}$ is the standard metric between word vectors.

Exercises

1. Build the term-document matrix with `CountVectorizer`. What is its shape? How sparse is it?
2. Switch to `TfidfVectorizer`. Which words gain weight? Which lose weight?
3. Compute cosine similarity between pairs of words from the same topic and across topics. Do the numbers match your intuition?

3.1.2 2) Latent Semantic Analysis — SVD and latent space

Goal: Apply Truncated SVD to reduce dimensionality and visualize the latent semantic structure.

Key ideas

- LSA factorizes the matrix $W = U\Sigma V^T$ and keeps only the top k singular values: $\hat{W} = U_k \Sigma_k V_k^T$.
- The rows of V_k are document embeddings; the rows of U_k (scaled by Σ_k) are word embeddings.
- Dimensionality reduction acts as denoising: synonymous words that never co-occur in the same document become close in the latent space.

Exercises

1. Apply `TruncatedSVD` with $k = 2$ and plot documents in the latent plane. Do the three topics separate cleanly?
2. Plot the word vectors in the same plane. Which words anchor each dimension?
3. Increase k to 5 and 10. How does explained variance change? Use the document embeddings to retrieve the top-3 documents for a free-text query.
4. **Reflection:** What does each singular value measure geometrically? Why does LSA struggle with polysemy (e.g. *star* = celebrity or astronomical object)?

3.1.3 3) Word2Vec on the toy corpus

Goal: Train a skip-gram Word2Vec model and compare its embeddings with LSA.

Key ideas

- Word2Vec trains a binary classifier: given a (target, context) pair, predict whether the context is real or noise-sampled.
- The objective $L = \sum_{(w,c):y=1} \log \sigma(\mathbf{c}^T\mathbf{w}) + \sum_{(w',c'):y=0} \log \sigma(-\mathbf{c}'^T\mathbf{w}')$ is maximized by gradient descent.
- The context window L controls what "context" means: small L captures syntactic relations, large L captures topical/semantic relations.

Exercises

1. Train Word2Vec (`sg=1`, `window=3`, `epochs=200`) on the toy corpus. Visualize the embeddings with PCA.
2. Compare side-by-side: for the same word pairs, are Word2Vec similarities higher/lower/more discriminative than LSA?
3. Try `sg=0` (CBOW) and `window=1` vs `window=5`. What changes in the clustering?
4. **Reflection:** Why does Word2Vec underperform on this 18-sentence corpus? What data regime favours LSA?

3.1.4 4) Pre-trained embeddings and analogy tasks

Goal: Load large-scale GloVe vectors and test the famous linear structure of word embeddings.

Key ideas

- Pre-trained GloVe vectors are trained on Wikipedia + Gigaword (~6 billion tokens, 400k vocabulary, 50–300 dimensions).
- The linear analogy structure **king** − **man** + **woman** ≈ **queen** emerges from training at scale.
- This structure does **not** appear in LSA or Word2Vec trained on small corpora.

Exercises

1. Load `glove-wiki-gigaword-50` with `gensim.downloader`. Solve the analogies: *king − man + woman*, *Paris − France + Italy*, *walk − walking + swimming*.
2. Try to solve analogies with your toy LSA vectors. What happens?
3. Find 3 word pairs where GloVe similarities agree with LSA, and 3 where they strongly disagree. What explains the disagreements?

3.1.5 5) Comparison and reflection

Goal: Synthesise the trade-offs between LSA and Word2Vec.

Property	LSA	Word2Vec
Method	SVD on count matrix	Neural skip-gram
Training	Closed-form	Gradient descent
Corpus size needed	Small OK	Needs billions of words
Polysemy handling	Poor (single vector)	Poor (single vector)
Analogy structure	Weak	Strong at scale
Interpretability	Moderate (SVD components)	Low
Matrix storage	$O(\|V\| \cdot D)$	$O(\|V\| \cdot d)$

Exercises

1. In which practical scenario would you choose LSA over Word2Vec? Give two concrete examples.
2. Both methods produce a **single vector per word** regardless of context. Why is this a fundamental limitation? What architecture overcomes it? (Hint: see `chapters/transformers.qmd`.)
3. The lecture notes describe PPMI as an alternative to raw counts before SVD. Implement PPMI weighting and compare LSA-PPMI similarity with raw-count LSA and Word2Vec.

Chapter 4

The nature of transformers

- Deep learning architecture based on **attention mechanisms**
- Weight the importance of different tokens in a sequence to **model long-range dependencies efficiently**

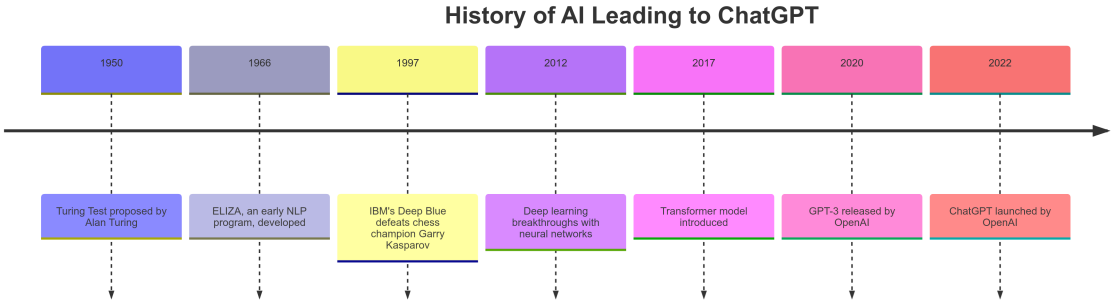
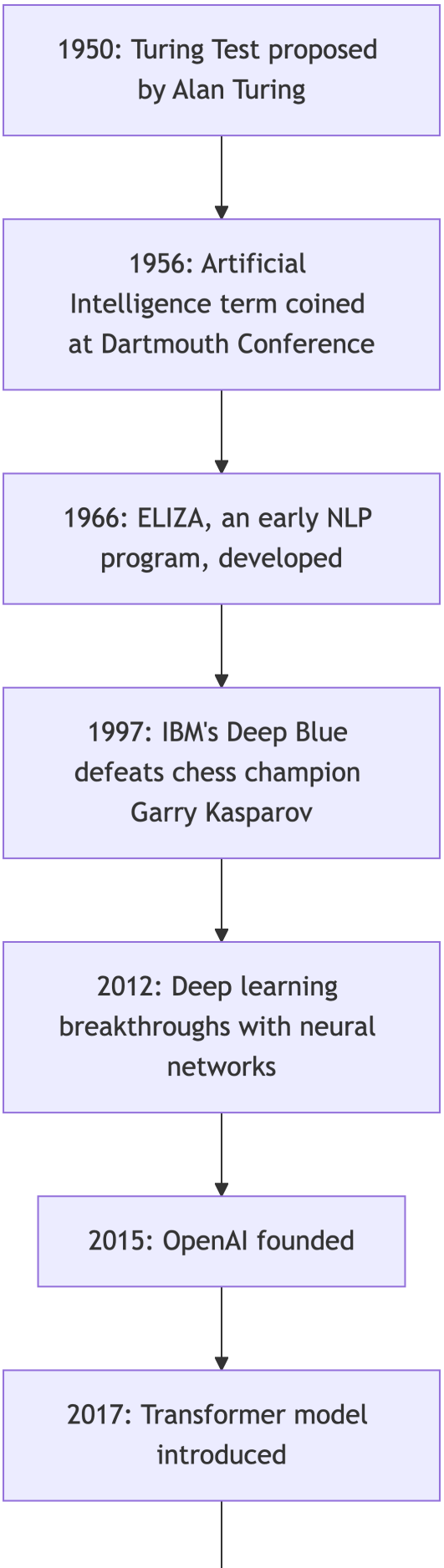


Figure 4.1: History of AI leading to chatGPT


1950: Turing Test proposed by Alan Turing
1956: Artificial Intelligence term coined at Dartmouth Conference
1966: ELIZA, an early NLP program, developed
1997: IBM's Deep Blue defeats chess champion Garry Kasparov
2012: Deep learning breakthroughs with neural networks
2015: OpenAI founded
2017: Transformer model introduced

4.1 The nature of transformers

A classical transformer is a function

$$T(w_{1:n}) = \begin{bmatrix} p_1 \\ \vdots \\ p_{|V|} \end{bmatrix}$$

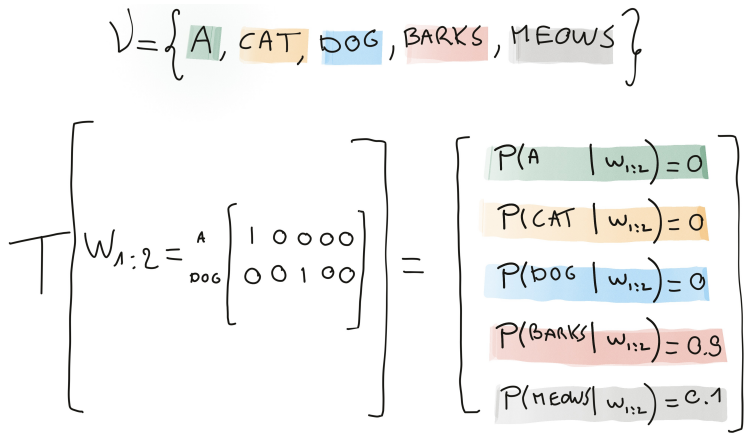


Figure 4.2: The nature of transformer

- $\mathcal{V}$ is a finite set (vocabulary) (Open AI uses 50,000 token as vocabulary)
- $w_{1:n}$ be a sequence of token, where each $w_i \in \mathcal{V}$ (In practice context varies from 1 to n, the blocksize)
- p_j is the probability of $w_{(j)}$ the jth token in $\mathcal{V}$

4.1.1 Remarks

- $w_{1:n}$ is usually coded a matrix of one-hot vectors with $w_{1:n} \in \mathbb{R}^{n \times |V|}$.
- For classical chatbot $p_j = P(w_{n+1} = w_{(j)} | w_{1:n})$ where $w_{(j)}$ is the jthe element of $\mathcal{V}$.

4.2 Optimized criteria

Transformers typically optimize different criteria depending on the task they are trained on.

4.2.1 Causal Language Modeling (CLM) / Autoregressive Loss

- **Used in:** GPT models
- **Objective:** Predict the next token given the previous tokens

The training set is a set of sequences (s_j) of n tokens $s_j = (w_1^j, w_2^j, \dots, w_n^j)$:

$$L_{CLM} = -\sum_j \sum_{t=1}^{n} \sum_{c \in V} \boldsymbol{I}_{w_t^j = c} \log P(w_t^j = c | w_{1:t-1}^j)$$

where $w_{1:t-1}^j$ denotes the tokens preceding position t in the sequence j

The criterion is a kind of pseudo log-likelihood.

If nothing is learned each token has the same probability of appearing: the largest possible value of the criterion is

$$-\log \frac{1}{|V|}$$

4.3 Masked Language Modeling (MLM)

- **Used in:** BERT models
- **Objective:** Predict randomly masked tokens in a sentence.

Let $M_j \subset \{1, \dots, n\}$ be the set of masked positions in the sequence (s_j):

$$L_{MLM} = -\sum_j \sum_{t \in M_j} \sum_{c \in V} \mathbb{I}_{w_t^j = c} \log P(w_t^j = c | w_{(1:n) \setminus M}^j)$$

4.4 Conditional Language Modeling / Translation Loss

- **Used in:** T5, BART
- **Objective:** Predict an output sequence given an input sequence (e.g., translation, summarization).

Given input sequence $w^j = (w_1^j, \dots, w_n^j)$ and output sequence $v^j = (v_1^j, \dots, v_m^j)$:

$$L_{Seq2Seq} = -\sum_j \sum_{t=1}^{m} \sum_{c \in V} \mathbb{I}_{v_t^j = c} \log P(v_m^j = c | v_{1:m-1}^j, w_{1:n}^j)$$

4.4.1 Other losses

Transformers exact objective functions depends on type of considered problems

4.5 Initial embedding

- The token are first embedded in a d dimensional space (see previous topic)
- the representation $x_i = Ew_i$ is independent of context.
- Let $X = x_{1:n} = w_{1:n} E^\top \in \mathbb{R}^{n \times d}$.

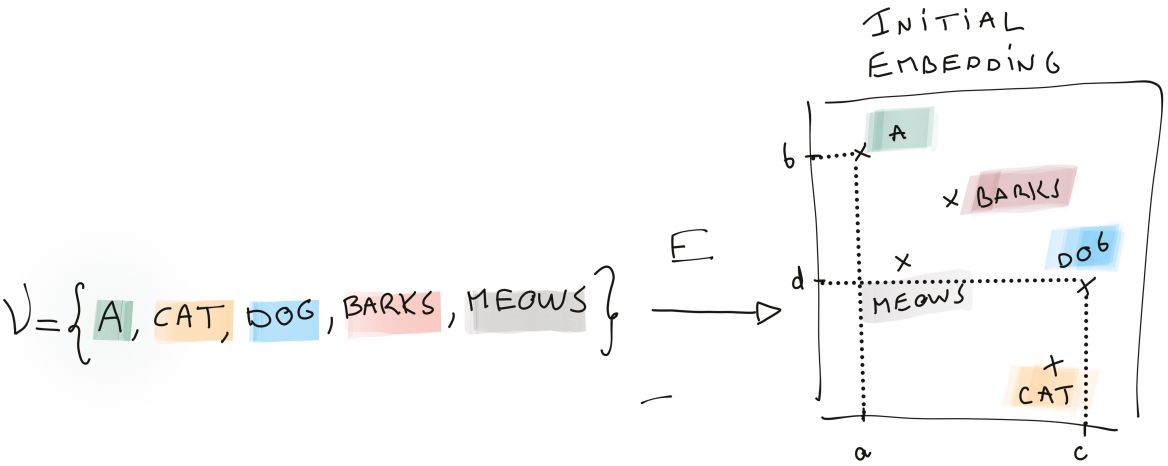


Figure 4.3: Initial embedding

4.6 A minimal self-attention architecture for decoder

Attention is a communication mecanism between the tokens of a sequence.

Self attention proposes a contextual representation h_i of x_i as a linear combination the sequence:

$$h_i = V.\sum_{j=1}^{n} \alpha_{ij} x_j$$

- h_i aggregates the past between query (current postion i) and key (other past position j)
  - $\sum_j \alpha_{ij} = 1$
- In decoder the future cannot communicate with the past
  - $\forall j > i, \alpha_{ij} = 0$
- the structure of the attention matrix is thus lower triangular in classical GPT decoder,
- it is not necessary to have a triangular attention matrix (i.e. sentiment analysis)
- V is learned matrix of parameters and can be seen a projection matrix modifying the embedding of the linear combination of sequence tokens. Let consider V as a square matrix $d \times d$.

4.7 Attention and matrix notation

- $\alpha_{ij} = \text{softmax}(x_i^T Q^T K x_j) = \frac{exp(x_i^T Q^T K x_j)}{\sum_{j=1}^{n} \exp(x_i^T Q^T K x_j)}$, where
  - Q is a matrix that modifies the embedding of the token we are looking for. Let consider V as a square matrix $d \times d$.
    - K is a matrix that modifies the embedding of the word we are comparing against. Let consider V as a square matrix $d \times d$.

4.7.1 Matrix notation

$$H = \begin{bmatrix} h_1^T \\ \vdots \\ h_n^T \end{bmatrix} = \text{Attention(Q,K,V,X)} = \text{softmax}(X(Q^T K)X^T)XV^T$$

4.7.2 Remark

In the original paper ("Attention is all you need", 2017) the notation is different and notes:

- XQ^T as Q,
- XK^T as K, and
- XV^T as V

4.8 Ilustration of self-Attention

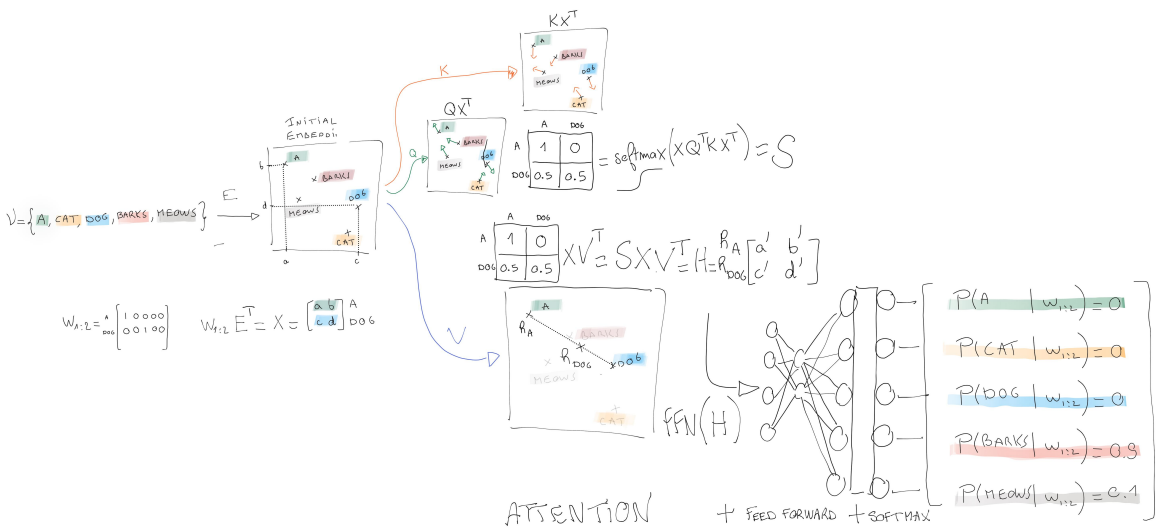


Figure 4.4: Self-attention block in auto-encoder

4.9 Interpretation of Q, K, V

- The query is modified version of an initial embedding $q_i = Qx_i$

- The Key is modified version of an initial embedding $k_i = Kx_i$
- Value is modified version of an initial embedding $v_i = Vx_i$, which should include contextual information since it is a transformation of a linear combination of the context tokens…

Attention Concept	Sometimes Similar to
Query	Target (What to focus on)
Key	Context (What to compare against, the source of information)
Value	Contextual embedding

4.10 Important details

The original paper introduces many add-ons, which much improve the transformers performances:

- **Positional encoding** for introducing the notion of order in the sequence
- FFN: **Feed Forward Neural layer**.
  - As the attention is a linear operation composed with a softmax, a Feed Forward Neural layer is added for making the function more flexible
- **Multi-blocks**: In the spirit of Deep learning blocks of attention and FNN are repeated…
- **Dropout layers** are used for prevenring overfitting (from "Dropout a simple way to prevent NN from overfitting" Hinton)
- Simple trick of **Skip connection** for stabilizing the gradient (from "Deep learning residuals for image recognition")
  - The deep learning structure tends to cause problems with gradient computation (vanishing or exploding gradient)
- In order to make tokens comparable, they are all **normalized** at the output of each blocks:
  - The classical z-transformed is applied (with Bayesian correction)
- **Multi-head attention**: computes multiple attention functions (heads) in parallel

4.11 Positional encoding

- Attention do not consider order of tokens !
- Incorporating information about the order of tokens is achieved by adding positional encodings to the input embeddings

The original transformer implementation uses:

- even indices:

$$PE_{\text{pos},2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

- odd indices:

$$PE_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

It ensures that each position is assigned a unique encoding

4.12 Feed Forward Neural Network

- **Attention mechanism operates linearly** to capture dependencies between tokens in a sequence
- Non-linearity enhance the model's expressive power

4.12.1 Structure of the FFN:

Each FFN consists of two linear transformations with a non-linear activation function in between:

1. First Linear Transformation: embedding of the input from the model's dimensionality d to a higher-dimensional space
2. Activation Function: A non-linear function is applied to introduce non-linearity
3. Second Linear Transformation: projects the output back to the original dimensionality

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2$$

where W_1 and W_2 are weight matrices, and b_1 and b_2 are bias vectors.

4.13 Skip connection (original paper "Deep Residual Learning for Image Recognition" 2015 by He et al.)

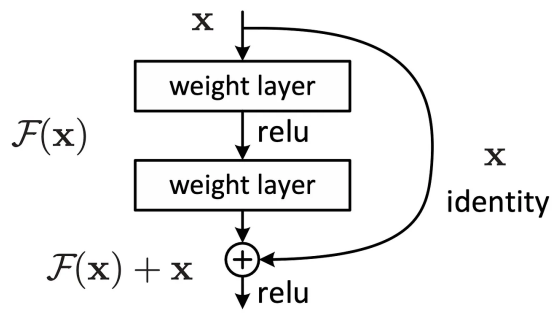


Figure 4.5: Skip connection

Skip connections address the vanishing (and exploding) gradient problems

- Consider a neural network layer with an input x and a desired underlying function $H(x)$.
- Incorporating a skip connection, the layer is restructured to model a residual $F(x) = H(x) - x$. Thus, the output of this layer becomes:

$$y = F(x) + x$$

4.13.1 Benefits:

- Enhanced Gradient Flow
- Faster convergence and higher accuracy

4.14 Multi-head attention

- Multi-Head Attention captures various aspects of the data.
- The mechanism computes multiple attention functions (heads) in parallel.
- Each head operates on linearly projected versions of the queries, keys, and values

Each head k computes the attention scores

$$\text{Attention}(Q_k, K_k, V_k) = \text{softmax}\left(\frac{XQ_k^T K_k X}{\sqrt{d_k}}\right) XV_k^T$$

where d_k is the dimension of the key vectors

4.14.1 Concatenation

The outputs of all heads are concatenated and projected through a final linear layer:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)P$$

The multi head matrix is $d \times d$, where - h is the number of head, - $d_k = d/h$ the size of the projection space of head k - P is a projection matrix combining the heads.

4.14.2 Benefits

- Diverse Representations: Each head can capture different features or relationships within the data, enabling the model to understand various aspects of the input (grammar, style, “meaning”…).
  - Parallel Processing: Multiple heads allow the model to process different parts of the sequence simultaneously, improving efficiency.
  - Enhanced Capacity: By attending to information from different subspaces, the model can capture complex patterns and dependencies.

4.15 Fine tuning

After a pretraining step that ressembles closely to a decoder…

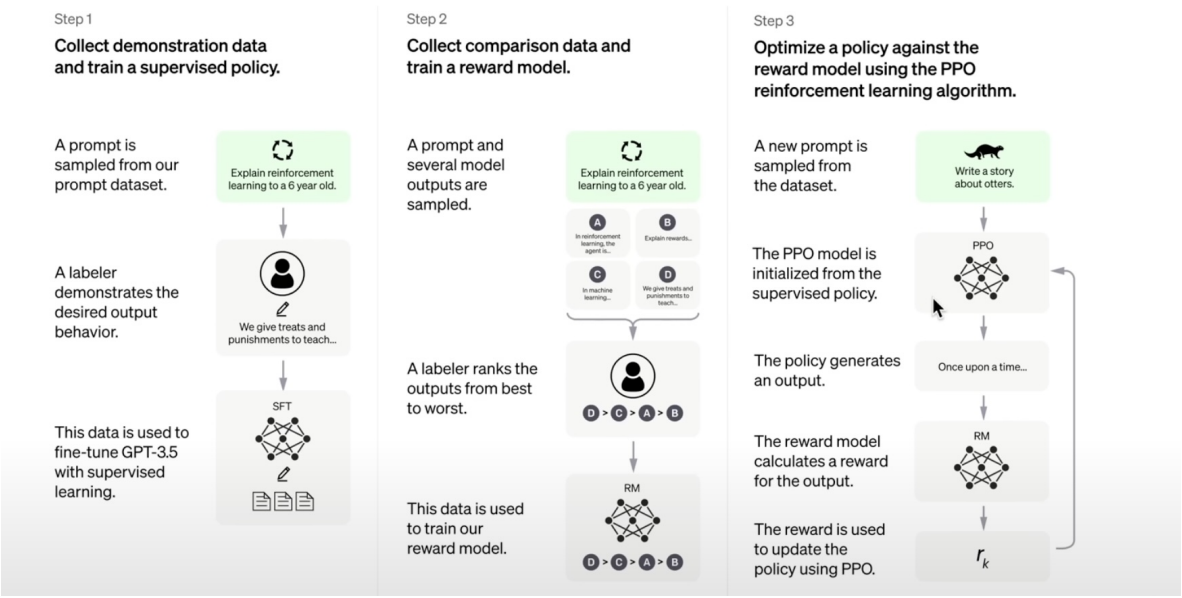


Figure 4.6: ChatGPT fine tuning (from A. Karpathy lecture)

Chapter 5

Exercises — GPT from scratch

These exercises follow the notebook `TP/gpt_dev.ipynb` step by step, building a small GPT character-level language model from scratch in PyTorch, following Karpathy's "Let's build GPT" series.

5.1 Links

- Tiny Shakespeare dataset
- Google Colab for the video
- GitHub repo for the video
- nanoGPT repo

5.2 Supplementary links

- Attention is All You Need — Vaswani et al. (2017)
- OpenAI GPT-3 paper
- OpenAI ChatGPT blog post

5.2.1 1) Data preparation and character-level tokenizer

Notebook cells: 1 – 8

Goal: Load a raw text corpus, build a character-level vocabulary, and encode the full dataset as a sequence of integers.

Key ideas

- The tiny Shakespeare dataset contains 1,115,394 characters drawn from 65 unique characters (letters, punctuation, newlines).
- A character-level tokenizer maps each character to an integer: `encode` turns a string into a list of integers, `decode` reverses it.
- The full corpus is stored as a `torch.Tensor` of dtype `torch.long`.
- The data is split 90 % train / 10 % validation *before* any model is built.

Exercises

1. Print the vocabulary (all 65 characters). Which characters appear? Is there anything surprising?
2. Encode the string `"Hello GPT"` and decode it back. Verify the round-trip.
3. What would change if you used a word-level tokenizer instead? Estimate the new vocabulary size and the new sequence length for the same corpus.
4. The split is done at position `int(0.9 * len(data))`. Why is it important *not* to shuffle the data before splitting?

5.2.2 2) Batch loading and the context window

Notebook cells: 9 – 12

Goal: Understand how a single `(input, target)` tensor pair encodes `block_size` training examples simultaneously.

Key ideas

- `block_size` is the maximum context length (number of tokens fed to the model at once).
- For a sequence of length `block_size + 1`, there are `block_size` overlapping `(context, target)` pairs: when the input is `x[:t+1]`, the target is `x[t+1]`.
- `get_batch` samples `batch_size` random starting positions and stacks the corresponding windows into tensors `(B, T)`.

Exercises

1. Run `get_batch('train')` and print `xb`, `yb`. For the first sequence in the batch, manually list all `(context, target)` pairs.
2. Why does `get_batch` sample starting positions from `range(len(data) - block_size)` rather than `range(len(data))`?
3. Change `block_size` to 4 and `batch_size` to 2 and rerun. How does this affect the number of training examples per batch?

5.2.3 3) Bigram language model

Notebook cells: 13 – 16

Goal: Implement the simplest possible language model as a baseline before introducing attention.

Key ideas

- A bigram model predicts the next character using only the current character: the token embedding table directly provides the logits for the next token.
- The loss is cross-entropy between the predicted distribution and the true next character.
- After 100 steps of AdamW (learning rate `1e-3`, batch size 32), the loss drops from 4.88 to 4.66, and the model generates readable (but random) character sequences.
- Generation uses multinomial sampling from the softmax distribution, appending one token at a time.

Exercises

1. What is the cross-entropy loss of a *uniform* random model over 65 characters? Compare with the initial loss of 4.88.
2. The `generate` method uses the whole sequence as context, but the bigram model only ever looks at the last token. Why is this not a problem here, but would be a problem for a Transformer?

3. Train for 1000 steps instead of 100. Does the loss continue to decrease? Generate a sample and comment on its quality.
4. Modify `generate` to use greedy decoding (argmax) instead of sampling. How does the output change?

5.2.4 4) The mathematical trick in self-attention

Notebook cells: 17 – 22

Goal: Understand weighted aggregation of past tokens via matrix multiplication, the foundation of self-attention.

Key ideas

- A token at position t should only communicate with positions $\leq t$ (causal constraint).
- The simplest aggregation: $x_t^{\text{bow}} = \frac{1}{t}\sum_{i\leq t} x_i$ (bag-of-words average).
- **Version 1** (loop): explicit `for` loop over positions — correct but slow.
- **Version 2** (matrix multiply): replace the loop with a lower-triangular matrix multiply. If `wei` is the $(T \times T)$ lower-triangular matrix normalised by row, then `wei @ x` computes all position averages in one operation.
- **Version 3** (softmax): initialise `wei = zeros`, fill future positions with `-inf`, then apply softmax — this is exactly the uniform-attention baseline.

Exercises

1. Verify that versions 1, 2 and 3 produce identical outputs using `torch.allclose`.
2. Print the weight matrix `wei` for version 2 (shape $T \times T$). What does each row represent?
3. What happens if you remove the masking step in version 3? What values does softmax assign to future positions?
4. In version 2, why must the rows be normalised (divided by their sum)? What would happen to the magnitude of `xbow` without normalisation?

5.2.5 5) Self-attention with Query, Key and Value

Notebook cells: 23 – 25

Goal: Replace the uniform average with data-dependent attention weights computed from learned projections.

Key ideas

- Each position produces three vectors from the input x: a **query** $q = Qx$ (what am I looking for?), a **key** $k = Kx$ (what do I contain?), and a **value** $v = Vx$ (what do I communicate?).
- Attention scores: $\text{wei} = qk^T/\sqrt{d_k}$ — the dot product between the query at position t and the keys at all positions $\leq t$.
- After masking and softmax, the output is a weighted sum of values: $\text{out} = \text{wei} \cdot v$.
- **Self-attention**: Q, K, V are all produced from the same source x.
- **Encoder vs decoder**: removing the triangular mask gives an encoder block (all positions visible); keeping it gives a decoder block (causal, used for language modelling).

Exercises

1. Print `wei[0]` (attention weights for the first sequence in the batch). Verify that the upper triangle is exactly 0. Which position attends most to which?
2. The scaling factor is $1/\sqrt{d_k}$ (`head_size**-0.5`). Cells 26–31 show why: compute `wei.var()` with and without the scaling. What happens to the softmax distribution when the variance is large?
3. Remove the causal mask (delete the `masked_fill` line). The model can now look into the future. Why is this valid for an encoder but not for a language model?
4. Change `head_size` to 1, then to 64. How does the shape of `out` change?

5.2.6 6) Scaled attention and numerical stability

Notebook cells: 26 – 31

Goal: Understand why the $1/\sqrt{d_k}$ scaling prevents the softmax from saturating.

Key ideas

- If q and k are unit-variance, then $q \cdot k$ has variance d_k — its standard deviation grows with dimension.
- Without scaling, the dot products become large in magnitude, softmax converges to a one-hot distribution ("peaky"), and gradients vanish.
- With scaling by $1/\sqrt{d_k}$, `wei` stays unit-variance regardless of `head_size`.

Exercises

1. Run `k.var()`, `q.var()`, `wei.var()` before and after applying `* head_size**-0.5`. Confirm that scaling restores unit variance.
2. Compare `torch.softmax(tensor * 1, dim=-1)` and `torch.softmax(tensor * 8, dim=-1)` for the vector `[0.1, -0.2, 0.3, -0.2, 0.5]`. Describe the difference.
3. At initialisation, why do we want the attention weights to be approximately uniform (not peaky)?

5.2.7 7) Layer normalisation

Notebook cells: 32 – 34

Goal: Implement Layer Normalisation and understand how it differs from Batch Normalisation.

Key ideas

- LayerNorm normalises each sample independently across its feature dimension (not across the batch).
- For input x of shape (B, C): $\hat{x} = (x - \mu_x)/\sigma_x$, then out $= \gamma\hat{x} + \beta$ with learned scale γ and shift β.
- After LayerNorm, `x[0,:].mean()` `0` and `x[0,:].std()` `1` for every sample.
- In the modern Transformer ("Pre-LN"), LayerNorm is applied *before* attention and feed-forward (not after as in the original paper).

Exercises

1. Run the `LayerNorm1d` example with `x = torch.randn(32, 100)`. Verify that `x[0,:].mean()` and `x[0,:].std()` are approximately 0 and 1.
2. Check `x[:,0].mean()` and `x[:,0].std()` (one feature across the batch). Why are these *not* normalised?

3. What is the difference between this and `nn.BatchNorm1d`? In which training scenarios does LayerNorm have an advantage?

5.2.8 8) Full GPT implementation

Notebook cells: 36 – 37

Goal: Assemble all components into a complete character-level GPT and train it.

Architecture

```
Token embedding  (vocab_size → n_embd)
Position embedding (block_size → n_embd)
     N × Block:
         Pre-LN → MultiHeadAttention (n_head heads of size n_embd/n_head) → residual
         Pre-LN → FeedForward (n_embd → 4·n_embd → n_embd, ReLU) → residual
Final LayerNorm
Linear projection  (n_embd → vocab_size)
```

Hyperparameters (notebook defaults)

Parameter	Value
`batch_size`	16
`block_size`	32
`n_embd`	64
`n_head`	4
`n_layer`	4
`learning_rate`	1e-3
`max_iters`	5000
`dropout`	0.0

Results: 0.21 M parameters — train loss 1.66 / val loss 1.82 after 5000 steps.

Exercises

1. Count the parameters in each component (`Head`, `MultiHeadAttention`, `FeedFoward`, `Block`). Verify the total of 0.21 M.
2. The `Block` applies LayerNorm *before* the sub-layer (`x + self.sa(self.ln1(x))`). Swap to Post-LN (`self.ln1(x + self.sa(x))`). Does training still converge?
3. Set `dropout = 0.2` and retrain. Does the gap between train loss and val loss decrease?
4. Double `n_embd` to 128 and `n_layer` to 6. How many parameters does the model have now? What train/val loss do you achieve?
5. Train on a different text corpus (a novel, your own text). How does the generated output change?